

Teori Recurrent Neural Network

Ryan Fadhilah Faizal Hakim¹

¹Sains Data, Fakultas Sains dan Teknologi, IKOPIN University

1 Pendahuluan

Recurrent Neural Network (RNN) adalah kelas jaringan saraf tiruan yang dirancang untuk memodelkan data berurutan (*sequential data*), yaitu data yang memiliki ketergantungan terhadap urutan waktu atau posisi. Berbeda dengan *feedforward neural network* biasa yang mengasumsikan setiap masukan saling independen, RNN memiliki mekanisme umpan balik melalui keadaan tersembunyi (*hidden state*) sehingga informasi dari langkah sebelumnya dapat memengaruhi prediksi pada langkah berikutnya.

RNN banyak digunakan pada pemrosesan bahasa alami, pengenalan suara, analisis deret waktu, prediksi harga, pemodelan sinyal, penerjemahan mesin, peringkasan teks, dan sistem rekomendasi berbasis urutan. Inti dari RNN adalah kemampuan untuk mempertahankan memori internal yang merangkum informasi dari masukan sebelumnya.

Misalkan tersedia suatu urutan masukan

$$\mathbf{x}_{1:T} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T), \quad (1)$$

dengan T menyatakan panjang urutan dan $\mathbf{x}_t \in \mathbb{R}^{d_x}$ adalah vektor masukan pada waktu ke- t . RNN memproses urutan tersebut satu per satu dan menghasilkan keadaan tersembunyi

$$\mathbf{h}_{1:T} = (\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_T), \quad (2)$$

dengan $\mathbf{h}_t \in \mathbb{R}^{d_h}$ menyimpan representasi ringkas dari informasi historis hingga waktu ke- t .

2 Motivasi dan Karakteristik Data Berurutan

Data berurutan memiliki sifat bahwa elemen saat ini sering bergantung pada elemen sebelumnya. Contohnya, dalam kalimat “saya sedang makan”, kata berikutnya lebih mungkin berupa objek makanan daripada kata acak. Dalam deret waktu ekonomi, nilai pada hari ini sering dipengaruhi oleh nilai pada hari-hari sebelumnya. Ketergantungan semacam ini sulit ditangkap oleh model yang tidak memiliki memori.

Secara umum, tujuan pemodelan urutan adalah mempelajari hubungan probabilistik

$$p(\mathbf{y}_{1:T} \mid \mathbf{x}_{1:T}), \quad (3)$$

atau dalam kasus pemodelan bahasa,

$$p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_1, x_2, \dots, x_{t-1}). \quad (4)$$

Persamaan tersebut menunjukkan bahwa probabilitas suatu elemen dapat bergantung pada seluruh elemen sebelumnya. RNN mencoba merepresentasikan konteks historis tersebut melalui hidden state $\mathbf{h}_t$.

3 Arsitektur Dasar RNN

Pada RNN sederhana, hidden state diperbarui menggunakan masukan saat ini dan hidden state sebelumnya. Bentuk umum persamaannya adalah

$$\mathbf{h}_t = \phi(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h), \quad (5)$$

dengan

- $\mathbf{x}_t \in \mathbb{R}^{d_x}$: vektor input pada waktu ke- t ,
- $\mathbf{h}_t \in \mathbb{R}^{d_h}$: hidden state pada waktu ke- t ,
- $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$: bobot dari input ke hidden state,
- $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$: bobot rekuren dari hidden state sebelumnya,
- $\mathbf{b}_h \in \mathbb{R}^{d_h}$: bias hidden state,
- $\phi(\cdot)$: fungsi aktivasi, misalnya tanh atau ReLU.

Keluaran RNN pada waktu ke- t biasanya dihitung sebagai

$$\mathbf{o}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \quad (6)$$

dan prediksi akhirnya adalah

$$\hat{\mathbf{y}}_t = g(\mathbf{o}_t), \quad (7)$$

dengan $g(\cdot)$ merupakan fungsi keluaran. Untuk klasifikasi multi-kelas, g sering berupa fungsi *softmax*:

$$\hat{y}_{t,k} = \frac{\exp(o_{t,k})}{\sum_{j=1}^K \exp(o_{t,j})}, \quad (8)$$

dengan K adalah jumlah kelas.

Hidden state awal $\mathbf{h}_0$ biasanya diinisialisasi sebagai vektor nol atau parameter yang dapat dipelajari:

$$\mathbf{h}_0 = \mathbf{0}. \quad (9)$$

4 Interpretasi Hidden State sebagai Memori

Persamaan (5) menunjukkan bahwa $\mathbf{h}_t$ bergantung pada $\mathbf{x}_t$ dan $\mathbf{h}_{t-1}$. Karena $\mathbf{h}_{t-1}$ sendiri bergantung pada $\mathbf{x}_{t-1}$ dan $\mathbf{h}_{t-2}$, maka secara rekursif diperoleh

$$\mathbf{h}_t = F(\mathbf{x}_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_1), \quad (10)$$

untuk suatu fungsi nonlinier F . Dengan demikian, hidden state dapat dipandang sebagai memori terkompresi dari seluruh riwayat masukan sampai waktu t .

Jika fungsi aktivasi diabaikan untuk memahami intuisi linear, misalkan

$$\mathbf{h}_t = \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1}. \quad (11)$$

Dengan substitusi berulang, diperoleh

$$\mathbf{h}_t = \sum_{i=1}^t \mathbf{W}_{hh}^{t-i} \mathbf{W}_{xh} \mathbf{x}_i + \mathbf{W}_{hh}^t \mathbf{h}_0. \quad (12)$$

Persamaan ini memperlihatkan bahwa kontribusi masukan lama $\mathbf{x}_i$ dikalikan oleh pangkat matriks rekuren $\mathbf{W}_{hh}^{t-i}$. Inilah salah satu alasan mengapa RNN dapat mengalami masalah gradien menghilang atau meledak.

5 Jenis-Jenis Pemetaan RNN

RNN dapat digunakan dalam beberapa skema pemetaan input-output:

1. **One-to-one**: kasus jaringan biasa, satu input menghasilkan satu output.
2. **One-to-many**: satu input menghasilkan urutan output, misalnya pembuatan deskripsi gambar.
3. **Many-to-one**: urutan input menghasilkan satu output, misalnya klasifikasi sentimen kalimat.
4. **Many-to-many sinkron**: setiap input pada waktu t menghasilkan output pada waktu yang sama, misalnya pelabelan kata.
5. **Many-to-many asinkron**: seluruh urutan input diproses dahulu, lalu model menghasilkan urutan output, misalnya penerjemahan mesin.

Untuk kasus *many-to-one*, keluaran dapat dihitung dari hidden state terakhir:

$$\hat{\mathbf{y}} = g(\mathbf{W}_{hy}\mathbf{h}_T + \mathbf{b}_y). \quad (13)$$

Untuk kasus *many-to-many*, keluaran dihitung pada setiap waktu:

$$\hat{\mathbf{y}}_t = g(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y), \quad t = 1, 2, \dots, T. \quad (14)$$

6 Fungsi Aktivasi dalam RNN

Fungsi aktivasi berperan memberikan nonlinieritas. Aktivasi yang umum digunakan adalah tanh:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad (15)$$

dengan rentang nilai $(-1, 1)$. Turunannya adalah

$$\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z). \quad (16)$$

Alternatif lain adalah ReLU:

$$\text{ReLU}(z) = \max(0, z), \quad (17)$$

dengan turunan

$$\frac{d}{dz} \text{ReLU}(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases} \quad (18)$$

Pada RNN sederhana, tanh sering lebih stabil karena membatasi nilai hidden state, sedangkan ReLU dapat mempercepat pelatihan tetapi berisiko menyebabkan aktivasi menjadi sangat besar.

7 Fungsi Loss

Fungsi loss bergantung pada jenis tugas. Untuk klasifikasi multi-kelas pada setiap waktu, fungsi *cross-entropy* adalah

$$\mathcal{L} = - \sum_{t=1}^T \sum_{k=1}^K y_{t,k} \log(\hat{y}_{t,k}), \quad (19)$$

dengan $y_{t,k}$ adalah label sebenarnya dalam bentuk *one-hot* dan $\hat{y}_{t,k}$ adalah probabilitas prediksi.

Untuk klasifikasi satu keluaran pada akhir urutan, loss dapat ditulis sebagai

$$\mathcal{L} = - \sum_{k=1}^K y_k \log(\hat{y}_k). \quad (20)$$

Untuk regresi deret waktu, loss yang umum adalah *mean squared error* (MSE):

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^T \|\mathbf{y}_t - \hat{\mathbf{y}}_t\|_2^2. \quad (21)$$

Jika terdapat N sampel pelatihan, loss empirisnya adalah

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}^{(n)}(\theta), \quad (22)$$

dengan $\theta = \{\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hy}, \mathbf{b}_h, \mathbf{b}_y\}$ adalah seluruh parameter model.

8 Pelatihan RNN dengan Backpropagation Through Time

RNN dilatih menggunakan *Backpropagation Through Time* (BPTT). Ide dasarnya adalah membuka RNN sepanjang waktu sehingga menjadi jaringan feedforward yang sangat dalam dengan parameter yang digunakan berulang pada setiap waktu.

Misalkan

$$\mathbf{a}_t = \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h, \quad (23)$$

sehingga

$$\mathbf{h}_t = \phi(\mathbf{a}_t). \quad (24)$$

Gradien loss terhadap parameter diperoleh dengan aturan rantai. Untuk bobot output,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{o}_t} \frac{\partial \mathbf{o}_t}{\partial \mathbf{W}_{hy}}. \quad (25)$$

Untuk parameter rekuren, gradiennya lebih kompleks karena $\mathbf{W}_{hh}$ memengaruhi semua hidden state berikutnya. Secara umum,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} \frac{\partial \mathbf{a}_t}{\partial \mathbf{W}_{hh}}. \quad (26)$$

Karena

$$\frac{\partial \mathbf{a}_t}{\partial \mathbf{W}_{hh}} \propto \mathbf{h}_{t-1}, \quad (27)$$

maka kontribusi gradien pada waktu t sangat dipengaruhi oleh hidden state sebelumnya.

Untuk melihat aliran gradien antar waktu, perhatikan turunan

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(\phi'(\mathbf{a}_t)) \mathbf{W}_{hh}. \quad (28)$$

Turunan hidden state jauh ke belakang adalah

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^t \text{diag}(\phi'(\mathbf{a}_i)) \mathbf{W}_{hh}, \quad k < t. \quad (29)$$

Produk matriks berulang inilah yang menyebabkan gradien dapat mengecil menuju nol atau membesar tak terkendali.

9 Vanishing dan Exploding Gradient

Masalah utama pada RNN sederhana adalah *vanishing gradient* dan *exploding gradient*. Jika norma produk matriks dalam BPTT lebih kecil dari satu, gradien akan mengecil secara eksponensial:

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| \leq \prod_{i=k+1}^t \left\| \text{diag}(\phi'(\mathbf{a}_i)) \mathbf{W}_{hh} \right\|. \quad (30)$$

Jika setiap faktor rata-rata bernilai kurang dari satu, maka gradien mendekati nol ketika jarak waktu $t - k$ membesar. Akibatnya, model sulit mempelajari dependensi jangka panjang.

Sebaliknya, jika norma produk matriks lebih besar dari satu, gradien dapat meningkat secara eksponensial dan menyebabkan pelatihan tidak stabil. Salah satu solusi praktis untuk exploding gradient adalah *gradient clipping*:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g}, & \|\mathbf{g}\|_2 \leq c, \\ c \frac{\mathbf{g}}{\|\mathbf{g}\|_2}, & \|\mathbf{g}\|_2 > c, \end{cases} \quad (31)$$

dengan $\mathbf{g}$ adalah gradien dan c adalah ambang maksimum norma gradien.

10 Long Short-Term Memory

Long Short-Term Memory (LSTM) adalah varian RNN yang dirancang untuk mengurangi masalah vanishing gradient melalui mekanisme gerbang (*gate*) dan jalur memori eksplisit yang disebut *cell state*. LSTM menggunakan tiga gerbang utama: forget gate, input gate, dan output gate.

Untuk input $\mathbf{x}_t$, hidden state sebelumnya $\mathbf{h}_{t-1}$, dan cell state sebelumnya $\mathbf{c}_{t-1}$, persamaan LSTM adalah

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f), \quad (32)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \quad (33)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c), \quad (34)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (35)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \quad (36)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \quad (37)$$

dengan $\sigma(\cdot)$ adalah fungsi sigmoid,

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad (38)$$

dan $\odot$ menyatakan perkalian elemen per elemen.

Interpretasi setiap komponen adalah sebagai berikut:

- $\mathbf{f}_t$ menentukan bagian memori lama yang dipertahankan.
- $\mathbf{i}_t$ menentukan seberapa banyak informasi baru dimasukkan.
- $\tilde{\mathbf{c}}_t$ adalah kandidat isi memori baru.
- $\mathbf{c}_t$ adalah cell state yang membawa informasi jangka panjang.
- $\mathbf{o}_t$ menentukan bagian memori yang diekspos sebagai hidden state.

Keunggulan utama LSTM adalah adanya persamaan

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (39)$$

yang menyediakan jalur aditif bagi gradien. Jalur ini membuat informasi jangka panjang lebih mudah dipertahankan dibandingkan pada RNN sederhana.

11 Gated Recurrent Unit

Gated Recurrent Unit (GRU) adalah varian RNN yang lebih sederhana daripada LSTM. GRU tidak memiliki cell state terpisah, tetapi menggunakan reset gate dan update gate. Persamaannya adalah

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z), \quad (40)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r), \quad (41)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h), \quad (42)$$

$$\mathbf{h}_t = (\mathbf{1} - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (43)$$

Update gate $\mathbf{z}_t$ mengatur seberapa banyak hidden state lama diganti oleh kandidat baru. Reset gate $\mathbf{r}_t$ mengatur seberapa besar informasi masa lalu digunakan dalam pembentukan kandidat hidden state.

12 Bidirectional RNN

Pada beberapa tugas, prediksi pada posisi tertentu dapat memanfaatkan konteks sebelum dan sesudah posisi tersebut. *Bidirectional RNN* memproses urutan dalam dua arah: maju dan mundur. Hidden state maju didefinisikan sebagai

$$\vec{\mathbf{h}}_t = \phi(\vec{\mathbf{W}}_{xh} \mathbf{x}_t + \vec{\mathbf{W}}_{hh} \vec{\mathbf{h}}_{t-1} + \vec{\mathbf{b}}_h), \quad (44)$$

sedangkan hidden state mundur adalah

$$\overleftarrow{\mathbf{h}}_t = \phi(\overleftarrow{\mathbf{W}}_{xh} \mathbf{x}_t + \overleftarrow{\mathbf{W}}_{hh} \overleftarrow{\mathbf{h}}_{t+1} + \overleftarrow{\mathbf{b}}_h). \quad (45)$$

Representasi gabungan biasanya diperoleh dengan konkatenasi:

$$\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]. \quad (46)$$

Bidirectional RNN cocok untuk tugas seperti pelabelan entitas bernama, analisis sentimen berbasis seluruh kalimat, dan pelabelan sekuens ketika seluruh input tersedia sejak awal.

13 Deep RNN dan Stacked RNN

RNN dapat dibuat lebih dalam dengan menumpuk beberapa lapisan rekuren. Jika terdapat L lapisan, hidden state lapisan ke- ℓ pada waktu t dapat ditulis sebagai

$$\mathbf{h}_t^{(\ell)} = \phi \left(\mathbf{W}_{xh}^{(\ell)} \mathbf{h}_t^{(\ell-1)} + \mathbf{W}_{hh}^{(\ell)} \mathbf{h}_{t-1}^{(\ell)} + \mathbf{b}_h^{(\ell)} \right), \quad (47)$$

dengan $\mathbf{h}_t^{(0)} = \mathbf{x}_t$. Keluaran akhir biasanya diambil dari lapisan terakhir:

$$\hat{\mathbf{y}}_t = g(\mathbf{W}_{hy} \mathbf{h}_t^{(L)} + \mathbf{b}_y). \quad (48)$$

Stacked RNN dapat mempelajari representasi yang lebih abstrak, tetapi lebih sulit dilatih dan membutuhkan regularisasi yang baik.

14 Regularisasi pada RNN

RNN rentan terhadap *overfitting*, terutama ketika jumlah parameter besar dan data terbatas. Beberapa teknik regularisasi yang umum digunakan adalah:

1. **Dropout**: menonaktifkan sebagian unit secara acak selama pelatihan.
2. **Recurrent dropout**: menerapkan dropout pada koneksi rekuren dengan hati-hati agar tidak merusak memori temporal.
3. **Weight decay**: menambahkan penalti norma parameter pada fungsi loss.
4. **Early stopping**: menghentikan pelatihan ketika performa validasi tidak membaik.
5. **Gradient clipping**: membatasi norma gradien untuk mencegah exploding gradient.

Dengan regularisasi L2, objektif pelatihan menjadi

$$\mathcal{J}_{\text{reg}}(\theta) = \mathcal{J}(\theta) + \lambda \|\theta\|_2^2, \quad (49)$$

dengan λ adalah koefisien regularisasi.

15 Optimisasi Parameter

Parameter RNN diperbarui menggunakan algoritma optimisasi berbasis gradien. Pada *gradient descent*, pembaruan parameter adalah

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{J}(\theta), \quad (50)$$

dengan η adalah *learning rate*. Pada praktik modern, optimisasi sering menggunakan Adam, yang menggabungkan momentum dan normalisasi adaptif. Untuk gradien $\mathbf{g}_t$, Adam menghitung

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (51)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad (52)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (53)$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad (54)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \quad (55)$$

16 Contoh Perhitungan Sederhana

Misalkan RNN skalar memiliki persamaan

$$h_t = \tanh(w_x x_t + w_h h_{t-1} + b), \quad (56)$$

dengan $w_x = 0.5$, $w_h = 0.8$, $b = 0$, $h_0 = 0$, dan urutan input $(x_1, x_2, x_3) = (1, 2, 1)$. Maka

$$h_1 = \tanh(0.5(1) + 0.8(0)) = \tanh(0.5) \approx 0.462, \quad (57)$$

$$h_2 = \tanh(0.5(2) + 0.8(0.462)) = \tanh(1.370) \approx 0.879, \quad (58)$$

$$h_3 = \tanh(0.5(1) + 0.8(0.879)) = \tanh(1.203) \approx 0.835. \quad (59)$$

Contoh ini menunjukkan bahwa nilai h_3 tidak hanya dipengaruhi oleh x_3 , tetapi juga oleh informasi historis yang terkandung dalam h_2 .

17 Kelebihan dan Keterbatasan RNN

Kelebihan utama RNN adalah kemampuannya memproses urutan dengan panjang variabel dan menggunakan parameter yang sama untuk setiap langkah waktu. Hal ini membuat RNN lebih efisien daripada model yang memerlukan parameter berbeda untuk setiap posisi.

Namun, RNN sederhana memiliki beberapa keterbatasan:

- sulit mempelajari dependensi jangka panjang karena vanishing gradient,
- pelatihan cenderung lambat karena pemrosesan urutan bersifat sekuensial,
- rentan terhadap exploding gradient,
- kurang efisien dibandingkan arsitektur paralel seperti Transformer untuk urutan panjang,
- membutuhkan penanganan khusus untuk padding, masking, dan panjang urutan yang berbeda.

18 Perbandingan RNN, LSTM, dan GRU

Aspek	RNN Sederhana	LSTM	GRU
Memori jangka panjang	Lemah	Kuat	Cukup kuat
Jumlah gerbang	Tidak ada	Forget, input, output	Update, reset
Kompleksitas	Rendah	Tinggi	Sedang
Jumlah parameter	Paling sedikit	Paling banyak	Lebih sedikit dari LSTM
Stabilitas pelatihan	Kurang stabil	Lebih stabil	Lebih stabil dari RNN sederhana
Cocok untuk	Urutan pendek	Dependensi panjang	Kompromi efisiensi dan performa

19 Aplikasi RNN

Beberapa aplikasi RNN antara lain:

1. **Pemodelan bahasa:** memprediksi kata atau karakter berikutnya berdasarkan konteks sebelumnya.
2. **Klasifikasi teks:** menentukan sentimen, topik, atau label dokumen.
3. **Penerjemahan mesin:** memetakan urutan kata dalam satu bahasa ke bahasa lain.
4. **Pengenalan suara:** mengubah sinyal audio menjadi teks.
5. **Prediksi deret waktu:** memprediksi nilai masa depan dari data historis.
6. **Anomali sekuensial:** mendeteksi pola tidak normal pada data sensor atau transaksi.
7. **Bioinformatika:** menganalisis urutan DNA, RNA, atau protein.

20 Kesimpulan

Recurrent Neural Network adalah arsitektur jaringan saraf yang dirancang untuk memproses data berurutan dengan mempertahankan hidden state sebagai bentuk memori. Persamaan dasarnya menggabungkan input saat ini dan hidden state sebelumnya, sehingga RNN mampu menangkap pola temporal. Namun, RNN sederhana menghadapi kendala besar berupa vanishing dan exploding gradient. Varian seperti LSTM dan GRU dikembangkan untuk mengatasi masalah tersebut melalui mekanisme gerbang yang mengatur aliran informasi.

Secara matematis, kekuatan sekaligus kelemahan RNN berasal dari penggunaan bobot rekuren yang berulang sepanjang waktu. Mekanisme ini memungkinkan pembelajaran dependensi temporal,

tetapi juga membuat gradien menjadi hasil perkalian banyak matriks selama BPTT. Oleh karena itu, pemilihan arsitektur, fungsi aktivasi, regularisasi, optimisasi, serta teknik stabilisasi seperti gradient clipping sangat penting dalam penerapan RNN.